OpenAI

July 28, 2021 Release

Introducing Triton: Open-source GPU programming for neural networks

View code ↗ Read documentation ↗



Listen to article 7:34 Share

We're releasing Triton 1.0, an open-source Python-like programming language which enables researchers with no CUDA experience to write highly efficient GPU code—most of the time on par with what an expert would be able to produce.

Why it matters

OpenAI

multiplication kernels that match the performance of cuBLAS—something that many GPU programmers can't do—in under 25 lines of code. Our researchers have already used it to produce kernels that are up to 2x more efficient than equivalent Torch implementations, and we're excited to work with the community to make GPU programming more accessible to everyone.

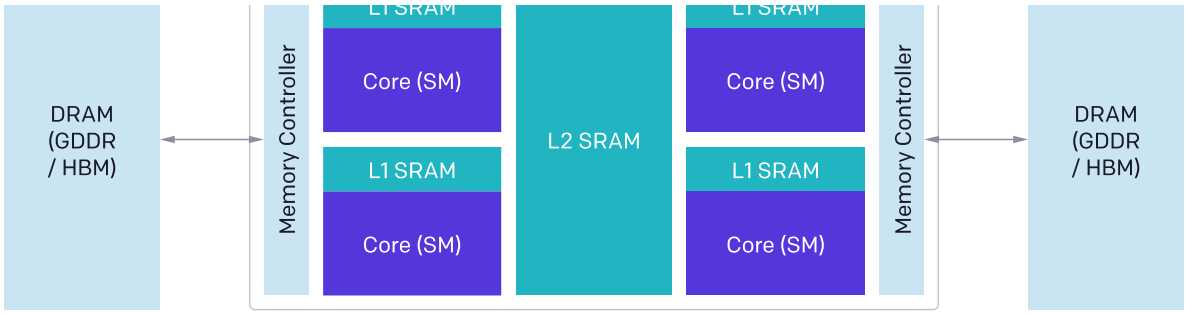
Novel research ideas in the field of Deep Learning are generally implemented using a combination of native framework operators. While convenient, this approach often requires the creation (and/or movement) of many temporary tensors, which can hurt the performance of neural networks at scale. These issues can be mitigated by writing specialized GPU kernels, but doing so can be surprisingly difficult due to the many intricacies of GPU programming.[1,2,3] And, although a variety of systems have recently emerged[4,5] to make this process easier, we have found them to be either too verbose, lack flexibility or generate code noticeably slower than our hand-tuned baselines. This has led us to extend and improve Triton[6], a recent language and compiler whose original creator now works at OpenAI.

The challenges of GPU programming

The architecture of modern GPUs can be roughly divided into three major components—DRAM, SRAM and ALUs—each of which must be considered when optimizing CUDA code:

- Memory transfers from DRAM must be *coalesced* into large transactions to leverage the large bus width of modern memory interfaces.
- Data must be manually stashed to SRAM prior to being re-used, and managed so as to minimize shared memory bank conflicts upon retrieval.
- Computations must be partitioned and scheduled carefully, both across and within Streaming Multiprocessors (SMs), so as to promote instruction/thread-level parallelism and leverage special-purpose ALUs (e.g., tensor cores).

OpenAI



Basic architecture of a GPU.

Reasoning about all these factors can be challenging, even for seasoned CUDA programmers with many years of experience. The purpose of Triton is to fully automate these optimizations, so that developers can better focus on the high-level logic of their parallel code. Triton aims to be broadly applicable, and therefore does not automatically schedule work across SMs -- leaving some important algorithmic considerations (e.g. tiling, inter-SM synchronization) to the discretion of developers.

	CUDA	TRITON
Memory Coalescing	Manual	Automatic
Shared Memory Management	Manual	Automatic
Scheduling (Within SMs)	Manual	Automatic
Scheduling (Across SMs)	Manual	Manual

Compiler optimizations in CUDA vs Triton.

Programming model

Out of all the Domain Specific Languages and JIT-compilers available, Triton is perhaps most similar to Numba: kernels are defined as decorated Python functions, and launched concurrently with different `program_id` 's on a grid of so-called *instances*. However, as shown in the code snippet below, the resemblance stops there: Triton exposes intra-instance parallelism via operations on *blocks*—small arrays whose dimensions are powers of two—rather than a Single

OpenAI

concurrency *within* CUDA thread blocks (e.g., memory coalescing, shared memory synchronization/conflicts, tensor core scheduling).

```
BLOCK = 512

# This is a GPU kernel in
# Different instances of
# function may run in par
@jit
def add(X, Y, Z, N):
    # In Numba/CUDA, each
    # instance itself uses
    # model, where instruc
    # parallel for differe
    tid = threadIdx.x
    bid = blockIdx.x
    # scalar index
    idx = bid * BLOCK + ti
    if id < N:
      # There is no pointe
      # Z,X,Y are dense te
      Z[idx] = X[idx] + Y[


...
grid = (ceil_div(N, BLOCK
block = (BLOCK,)
add[grid, block](x, y, z,
```

```
BLOCK = 512

# This is a GPU kernel in
# Different instances of
# function may run in par
@jit
def add(X, Y, Z, N):
    # In Triton, each kern
    # executes block opera
    # single thread: there
    # analogous to threadI
    pid = program_id(0)
    # block of indices
    idx = pid * BLOCK + ar
    mask = idx < N
    # Triton uses pointer
    # rather than indexing
    x = load(X + idx, mask
    y = load(Y + idx, mask
    store(Z + idx, x + y,


...
grid = (ceil_div(N, BLOCK
# no thread-block
add[grid](x, y, z, x.shap
```

Vector addition in Triton.

While this may not be particularly helpful for embarrassingly parallel (i.e., element-wise) computations, it can greatly simplify the development of more complex GPU programs.

Consider for example the case of a fused softmax kernel (below) in which each instance normalizes a different row of the given input tensor $X \in \mathbb{R}^{M \times N}$. Standard CUDA implementations of this parallelization strategy can be challenging to write, requiring explicit synchronization between threads as they concurrently reduce the same row of X. Most of this complexity goes away with Triton, where each kernel instance loads the row of interest and normalizes it sequentially using NumPy-like primitives.

Python

```python
import triton
import triton.language as tl

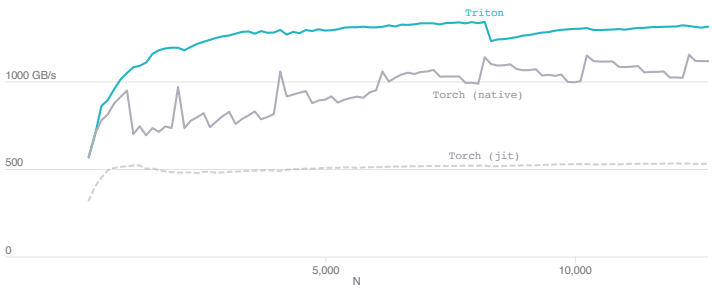
@triton.jit
def softmax(Y, stride_ym, stride_yn, X, stride_xm, stride_xn, M, N):
    # row index
```

OpenAI

```
    # this specific kernel only works for matrices that
    # have less than BLOCK_SIZE columns
    BLOCK_SIZE = 1024
    n = tl.arange(0, BLOCK_SIZE)
    # the memory address of all the elements
    # that we want to load can be computed as follows
    X = X + m * stride_xm + n * stride_xn
    # load input data; pad out-of-bounds elements with 0
    x = tl.load(X, mask=n < N, other=-float('inf'))
    # compute numerically-stable softmax
    z = x - tl.max(x, axis=0)
    num = tl.exp(z)
    denom = tl.sum(num, axis=0)
    y = num / denom
    # write back to Y
    Y = Y + m * stride_ym + n * stride_yn
    tl.store(Y, y, mask=n < N)
```

Note that the Triton JIT treats X and Y as *pointers* rather than tensors; we felt like retaining low-level control of memory accesses was important to address more complex data structures (e.g., block-sparse tensors).

Importantly, this particular implementation of softmax keeps the rows of *X* in SRAM throughout the entire normalization process, which maximizes data reuse when applicable (~<32K columns). This differs from PyTorch's internal CUDA code, whose use of temporary memory makes it more general but significantly slower (below). The bottom line here is not that Triton is inherently better, but that it simplifies the development of specialized kernels that can be much faster than those found in general-purpose libraries.



A100 performance of fused softmax for M=4096.

The lower performance of the Torch (v1.9) JIT highlights the difficulty of automatic CUDA code generation from sequences of high-level tensor operations.

OpenAI

```python
@torch.jit.script
def softmax(x):
    x_max = x.max(dim=1)[0]
    z = x - x_max[:, None]
    numerator = torch.exp(x)
    denominator = numerator.sum(dim=1)
    return numerator / denominator[:, None]
```

Fused softmax with the Torch JIT.

Matrix multiplication

Being able to write fused kernels for element-wise operations and reductions is important, but not sufficient given the prominence of matrix multiplication tasks in neural networks. As it turns out, Triton also works very well for those, achieving peak performance with just ~25 lines of Python code. On the other hand, implementing something similar in CUDA would take a lot more effort and would even be likely to achieve lower performance.

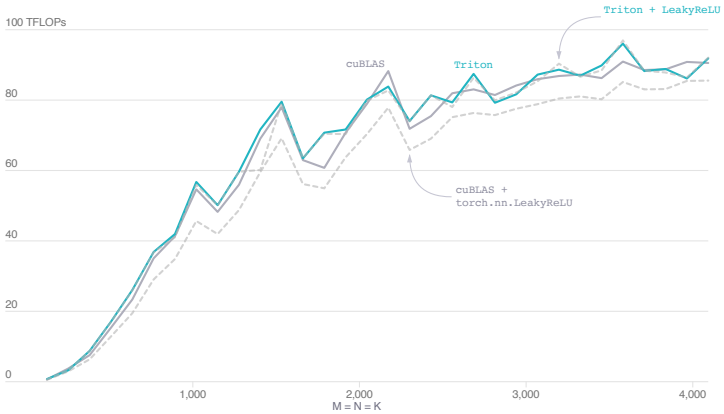
Python

```python
@triton.jit
def matmul(A, B, C, M, N, K, stride_am, stride_ak,
           stride_bk, stride_bn, stride_cm, stride_cn,
           **META):
    # extract metaparameters
    BLOCK_M, GROUP_M = META['BLOCK_M'], META['GROUP_M']
    BLOCK_N = META['BLOCK_N']
    BLOCK_K = META['BLOCK_K']
    # programs are grouped together to improve L2 hit rate
    _pid_m = tl.program_id(0)
    _pid_n = tl.program_id(1)
    pid_m = _pid_m // GROUP_M
    pid_n = (_pid_n * GROUP_M) + (_pid_m % GROUP_M)
    # rm (resp. rn) denotes a range of indices
    # for rows (resp. col) of C
    rm = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
    rn = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
    # rk denotes a range of indices for columns
    # (resp. rows) of A (resp. B)
    rk = tl.arange(0, BLOCK_K)
    # the memory addresses of elements in the first block of
    # A and B can be computed using numpy-style broadcasting
    A = A + (rm[:, None] * stride_am + rk[None, :] * stride_ak)
    B = B + (rk [:, None] * stride_bk  + rn[None, :] * stride_bn)
    # initialize and iteratively update accumulator
    acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)
    for k in range(K, 0, -BLOCK_K):
```

Matrix multiplication in Triton.

OpenAI

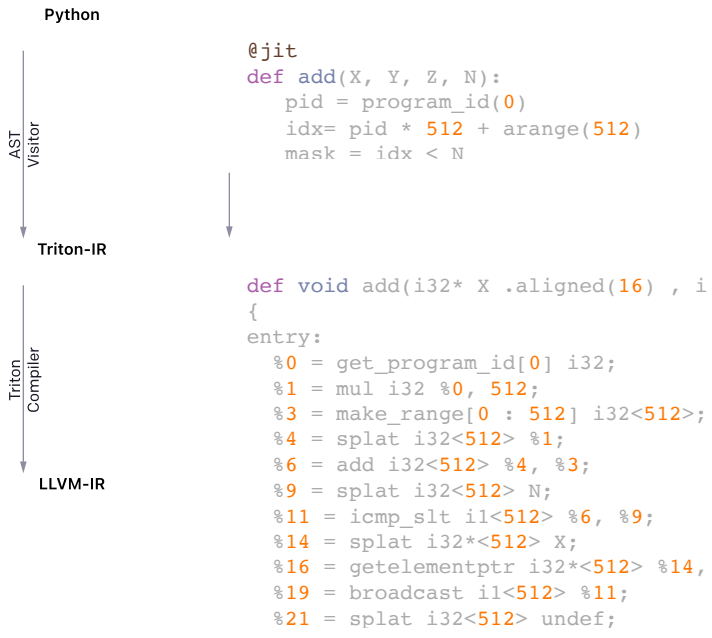
fused transformations of their inputs (e.g., slicing) and outputs (e.g., Leaky ReLU). Without a system like Triton, non-trivial modifications of matrix multiplication kernels would be out-of-reach for developers without exceptional GPU programming expertise.



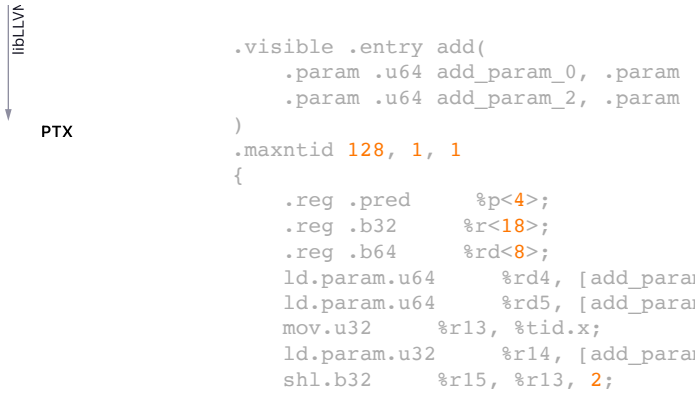
V100 tensor-core performance of matrix multiplication with appropriately tuned values for BLOCK_M, BLOCK_N, BLOCK_K, GROUP_M.

High-level system architecture

The good performance of Triton comes from a modular system architecture centered around Triton-IR, an LLVM-based intermediate representation in which multi-dimensional blocks of values are first-class citizens.



OpenAI

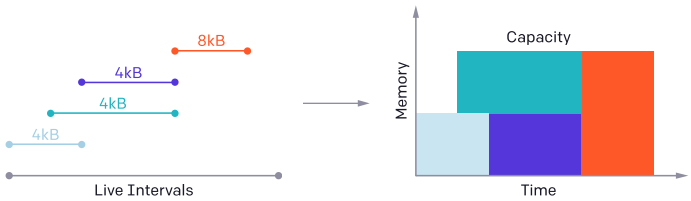


High-level architecture of Triton.

The `@triton.jit` decorator works by walking the Abstract Syntax Tree (AST) of the provided Python function so as to generate Triton-IR on-the-fly using a common SSA construction algorithm.[8] The resulting IR code is then simplified, optimized and automatically parallelized by our compiler backend, before being converted into high-quality LLVM-IR—and eventually PTX—for execution on recent NVIDIA GPUs. CPUs and AMD GPUs are not supported at the moment, but we welcome community contributions aimed at addressing this limitation.

Compiler backend

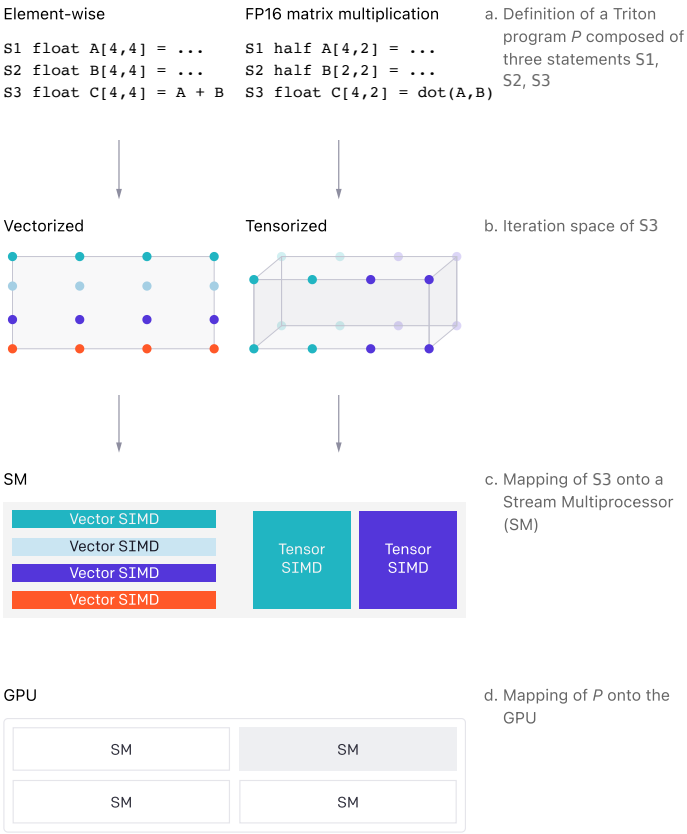
We have found that the use of blocked program representations via Triton-IR allows our compiler to automatically perform a wide variety of important program optimizations. For example, data can be automatically stashed to shared memory by looking at the operands of computationally intensive block-level operations (e.g., `tl.dot`)—and allocated/synchronized using standard liveness analysis techniques.



The Triton compiler allocates shared memory by analyzing the live range of block variables used in computationally intensive operations.

On the other hand, Triton programs can be efficiently and automatically parallelized both (1) across SMs by executing different

adequately across different SIMD units, as shown below.



Automatic parallelization in Triton. Each block-level operation defines a blocked iteration space that is automatically parallelized to make use of the resources available on a Streaming Multiprocessor (SM).

Contributing

We intend for Triton to become a community-driven project. Feel free to fork our repository on GitHub!

If you're interested in joining our team and working on Triton & GPU kernels, we're hiring!

Community & Collaboration Software & Engineering

References

OpenAI

2 Kerr, A. (2020). Developing CUDA kernels to push Tensor Cores to the Absolute Limit on NVIDIA A100. *URL https://developer.nvidia.com/gtc/2020/video/s21745-vid*. ↩

3 Yan, D., Wang, W., & Chu, X. (2020, May). Demystifying tensor cores to optimize half-precision matrix multiply. In *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE. ↩

4 NVIDIA CUTLASS ↩

5 Apache TVM ↩

6 Tillet, P., Kung, H. T., & Cox, D. (2019, June). Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* (pp. 10-19). ↩

7 Lin, Y. & Grover, V. (2018). Using CUDA Warp-Level Primitives. *URL https://developer.nvidia.com/blog/using-cuda-warp-level-primitives/*. ↩

8 Braun, M., Buchwald, S., Hack, S., Leißa, R., Mallon, C., & Zwinkau, A. (2013, March). Simple and efficient construction of static single assignment form. In *International Conference on Compiler Construction* (pp. 102-122). Springer, Berlin, Heidelberg. ↩

Author

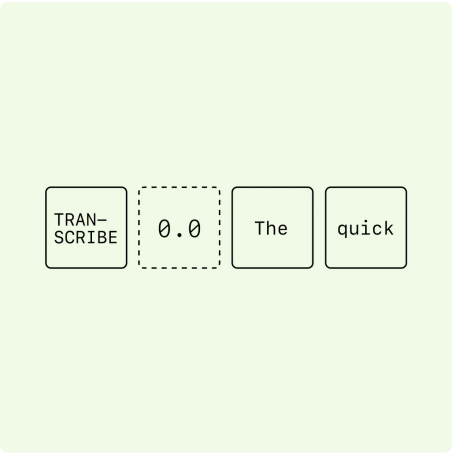
Philippe Tillet

Acknowledgments

Da Yan (HKUST), DeepSpeed (Microsoft), Anthropic

Related articles

View all



Introducing Whisper





Scaling Kubernetes to 7,500 nodes

OpenAI

Publication Jun 9, 2022

OpenAI © 2015–2026 Your privacy choices